Diligent Code Studio

Community Edition User Manual

Version 0.5.5 - Help Button and Built-in PDF Manual

Purpose	A local-first development workbench for editing code, managing projects, using AI assistance, checking dependencies, running builds, and preparing releases.
Audience	Developers, IT administrators, power users, and Diligent Software Services project maintainers.
Platform	Windows-focused desktop workflow with cross-platform source support through Tauri, React, TypeScript, Rust, and Vite.

This manual explains how to navigate the application, use the Workspace Menu, configure setup dependencies, work with the editor, access AI help, use Git, run diagnostics, and package releases.

Diligent Software Services

Table of Contents

1. Overview

What Diligent Code Studio is

Diligent Code Studio is a desktop software-building workbench designed to keep the common development tasks in one place. It combines project navigation, editing, AI assistance, dependency checks, Git tools, diagnostics, release packaging, and logs.

The application is designed around a local-first workflow. Your project files remain on your computer. AI features are optional and must be configured in Settings before use.

Primary goals

- Make project setup easier by checking required tools and dependencies.
- Give the user a clean editor with project navigation and file actions.
- Provide compact AI help on every screen without taking over the workspace.
- Help users run terminal commands, diagnose errors, manage Git, and package releases.
- Keep the interface beginner-friendly while preserving advanced project controls.

2. First Launch

Recommended first steps

- Open the application from the installed shortcut or compiled executable.
- Choose a workspace folder from the left Workspace panel.
- Open Setup & Dependencies and select Check Again to verify tools.
- Open Settings to choose theme, compact mode, terminal shell, and AI provider.
- Use the Help button in the top-right corner to open this PDF manual inside the software.

Important note

The development command npm run tauri:dev is only for testing from source. The installed desktop version should run as a normal application and should not require an npm window to remain open.

3. Main Layout

Application zones

- Top app bar - shows the application name, project health indicators, Setup, Help, and AI Help controls.
- Left sidebar - workspace path, file actions, filter box, and project file tree.
- Top Workspace Menu - main navigation buttons for Templates, Setup, Editor, AI, Search, Terminal, Git, Problems, Release, Tool Registry, Project Dashboard, Logs, and Settings.
- Main content area - shows the selected screen.
- Compact AI Help - floating assistant available on every screen.

Why the terminal is not global

The terminal is intentionally no longer open on every screen. It is available from the Terminal page so normal screens stay clean and focused.

4. Help Button and PDF Manual

Opening the manual

Version 0.5.5 adds a Help button in the top-right app bar. Selecting Help opens this PDF manual inside a built-in help viewer overlay.

Help viewer controls

- Open PDF - opens the manual PDF in a new viewer window or browser-capable view if supported by the system WebView.
- Close - returns to the current Diligent Code Studio screen.
- Scroll - use the PDF viewer scroll controls to move through this manual.

Where the PDF is stored

The manual is included with the application files under public/manuals in the source package and is copied into the built frontend output during the normal Vite build process.

5. Workspace Menu

Purpose

The Workspace Menu is the main navigation area. It stays in its own fixed top space and does not horizontally scroll. Buttons wrap naturally when needed.

Drag-and-drop ordering

- Drag a Workspace Menu button left or right to reorder it.
- Drop on the left half of another button to place it before that button.
- Drop on the right half to place it after that button.
- The new order is saved to preferences automatically.
- Use Reset Menu Order in Settings to restore the default order.

Default order

Templates is designed to start farthest left by default, followed by Setup & Dependencies, Editor, AI, Find/Search, Terminal, Git, Problems, Release, Tool Registry, Project Dashboard, Logs, and Settings.

6. Workspace and File Explorer

Choosing a workspace

Use the Choose Folder button in the left sidebar to select the project folder you want to work on. Diligent Code Studio scans the folder and displays files and folders in the tree.

File actions

- New File creates a file in the selected folder or current target directory.
- New Folder creates a folder in the selected target directory.
- Rename renames the currently selected file or folder.
- Delete removes the selected file or folder after confirmation.
- Refresh reloads the workspace tree.

Filtering files

Use the filter box above the file tree to quickly find files or folders by name.

7. Editor

Opening and editing files

Select a file in the left file tree to open it in the editor. Open files appear as tabs. Unsaved changes are marked and can be saved from the toolbar.

Editor toolbar

- Save - saves the active file.
- Save As - writes the active file to a new location.
- Format - formats supported document types when available.
- SHA - creates a SHA-256 hash for the active file.
- AI Help - opens or minimizes compact AI help.

Status strip

The footer under the editor shows the active file path, detected language, line count, cursor location, save status, and hash status.

8. Compact AI Help

Purpose

Compact AI Help is available on every screen but no longer takes up a large permanent panel. It can be minimized to a small bottom-right AI Help button or expanded when needed.

AI quick actions

- Explain - prepares a prompt to explain selected code or the active file.
- Bugs - prepares a prompt to review the current file for problems.
- Errors - focuses the AI context on diagnostics and problems.
- Terminal - focuses the AI context on terminal output.
- Git - focuses the AI context on Git status.
- Navigate - asks for help deciding what to do next on the current screen.

Privacy reminder

AI is optional. Configure the provider in Settings. Review any prompt before sending project content to an external provider.

9. Setup & Dependencies

Purpose

Setup & Dependencies checks whether the computer has the tools needed for common software projects.

Typical tools checked

- Git and GitHub CLI for source control and GitHub workflows.
- Node.js and npm for JavaScript, TypeScript, React, and Vite projects.
- Rust and Cargo for Tauri desktop builds.
- Visual Studio Build Tools for Windows native build support.
- WebView2 Runtime for Windows desktop webview support.
- Ollama for optional local AI models.

Installer results

If winget reports that a package is already installed or up to date, Diligent Code Studio treats that as success instead of a failure.

10. Tool Registry

Purpose

Tool Registry is the app catalog for registered tools, commands, templates, scripts, models, and utilities. It is not the Windows Registry.

How to use it

- Review built-in tool definitions.
- Add custom tools or scripts used in your workflow.
- Enable or disable tools depending on the project.
- Keep notes about where each tool is installed or how it is used.

11. Terminal

Purpose

The Terminal page lets you run project commands from the selected workspace. It is intentionally dedicated to one screen so it does not crowd every other screen.

Common commands

```
npm ci
npm run build
npm run tauri:dev
npm run tauri:build
git status
```

External terminal

Use Open Terminal to launch an external terminal window in the current project folder.

12. Git

Purpose

The Git page helps you review repository state, stage changes, commit work, create tags, and understand current branch status.

Recommended GitHub setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
gh auth login
gh auth status
```

When GitHub CLI is installed but not detected

Close and reopen Diligent Code Studio so the application receives the updated PATH. If needed, confirm the GitHub CLI path is included in your user PATH.

13. Problems and Diagnostics

Purpose

The Problems page runs diagnostics and collects build or syntax issues into a more readable view. It is useful after TypeScript, Vite, Rust, Cargo, or build errors.

How to use diagnostics

- Open Problems.
- Run diagnostics for the current workspace.
- Review the file, line, column, and message for each problem.
- Use Compact AI Help with the Errors context to ask for a fix plan.

14. Release Builder

Purpose

The Release Builder helps prepare project release packages after the project has been built.

Recommended build sequence

```
npm ci
npm run build
```

```
npm run tauri:build
```

Release outputs

- Compiled application or installer artifacts.
- ZIP package when supported by the build environment.
- SHA-256 checksum file.
- Release notes file.
- Copied bundle files for easier distribution.

15. Settings

Major settings areas

- Appearance - theme and compact mode.
- Editor - font size, word wrap, and save behavior.
- Workspace - startup folder and workspace memory options.
- AI - provider, model, endpoint, context, and confirmation settings.
- Terminal - preferred shell.
- Workspace Menu - reset menu order to defaults.

Resetting the menu

Use Reset Menu Order in Settings to restore the default Workspace Menu order. The reset control is intentionally kept out of the Workspace Menu itself.

16. Recommended Workflow

Daily workflow

- Open Diligent Code Studio.
- Choose or refresh the workspace.
- Open files from the left file tree.
- Edit and save changes.
- Use Problems to diagnose errors.
- Use Compact AI Help to explain errors or review code.
- Use Git to stage and commit changes.
- Use Release Builder when ready to package.

Testing this project from source

```
npm ci
npm run build
npm run tauri:dev
```

17. Troubleshooting

GitHub CLI installed but gh is not recognized

Close and reopen PowerShell and Diligent Code Studio. If the command still fails, confirm GitHub CLI is installed under C:\Program Files\GitHub CLI and add that folder to PATH.

npm window opens in installed version

The installed production app should not require an npm window. Confirm that your shortcut points to the compiled Diligent Code Studio executable and not to a development script or npm command.

Workspace menu is not in the desired order

Drag the menu buttons to reorder them. Use Settings, Reset Menu Order to return to the default order.

AI does not answer

Open Settings and configure OpenAI or Ollama. If using Ollama, verify the Ollama app is running and the configured model is available.

18. Keyboard and Usage Tips

Useful habits

- Keep one project per workspace when possible.
- Run Setup & Dependencies after installing new developer tools.
- Use Problems before asking AI about build errors so the assistant has better context.
- Use Git commits after each working milestone.
- Create a release package only after npm run build and npm run tauri:build pass.

Safety

Review file operations before deleting or overwriting files. Review AI responses before inserting generated content into source files.

Diligent Code Studio

Cumulative User Manual Addendum - v0.6.1

Manual Policy

Starting with v0.6.1, the built-in PDF manual is treated as a cumulative living document. The original v0.5.5 manual pages are preserved at the front of this PDF. New versions add addendum pages after the original manual instead of rewriting or replacing the beginning of the manual.

This keeps early setup, navigation, and core workflow instructions stable while still documenting new features, fixes, and behavior changes in later releases.

Manual Area	Policy
Original core manual	Keep unchanged at the beginning of the cumulative PDF.
Version updates	Append a new version addendum section for each release.
App Manual button	Open the stable cumulative file: manuals/DiligentCodeStudio_UserManual.pdf.
Versioned archive	Keep versioned/manual archive copies in docs/manuals when practical.

Version Addendum Index

Version	Focus	Key Additions
v0.5.6	Project-Aware AI	Added project-aware AI actions including Ask AI About This Project, Explain Current File, Find Bugs, Improve This Code, Generate Missing File, Create README, Create Installer Script, and Summarize Project.
v0.5.7	Version Sync	Synced visible app, package, Tauri, Cargo, and manual version references to prevent older visible version labels from remaining in the UI.
v0.5.8	Quality and Reliability	Added TypeScript checking, formatting configuration, validation scripts, Vite production optimization, GitHub Actions quality checks, and better Rust error handling.
v0.5.9	Security and Testing Foundation	Added security checks, smoke/unit test scripts, Rust test hooks, input validation, session-only OpenAI API key handling, and testing/security documentation.
v0.6.0	AI Help Cleanup	Reduced AI entry points to one upper-right AI Help button, minimized AI Help by default, separated AI Help responses from the main AI Coding Assistant output, and renamed Manual separately from AI Help.
v0.6.1	Cumulative Manual Policy	Changed manual handling so the original core manual remains intact and later release notes are appended as addenda. The app opens a stable cumulative manual path.

v0.5.6 - Project-Aware AI Assistant

Diligent Code Studio added a project-aware AI workflow so AI prompts can include more useful context than a simple text box. The assistant can now reason over the current screen, workspace path, detected project type, file and folder summary, active file preview, open files, Git status, diagnostics, terminal output, and release/build status when that context is available.

Primary buttons added:

- Ask AI About This Project
- Explain Current File
- Find Bugs
- Improve This Code
- Generate Missing File
- Create README
- Create Installer Script
- Summarize Project

Recommended use: use quick AI Help for navigation and basic screen guidance. Use the AI Coding Assistant workspace for longer coding results, generated files, readme drafts, installer scripts, and project summaries.

v0.5.7 - Version Sync Fix

This release fixed mismatched version labels. Package metadata, Tauri metadata, Cargo metadata, visible UI labels, and manual references should be updated together during each release. If the installed app still shows an older version after a new build, uninstall the previous install and verify the shortcut points to the compiled Diligent Code Studio executable.

v0.5.8 - Quality and Reliability Upgrade

This release added a more formal project quality gate. The recommended validation command became npm run validate, which runs TypeScript checks and the available test set. Production builds use Vite optimization and chunk splitting for key libraries such as React and Monaco.

Recommended validation flow:

```
npm ci
npm run validate
npm run build
npm run tauri:build
```

v0.5.9 - Security and Testing Foundation

This release added a stronger starter security and testing baseline. It added smoke, unit, security, and Rust test scripts, plus a source scan for obvious hardcoded secrets. It also added backend terminal command input validation and changed OpenAI API keys to session-only handling so they are not saved into persistent local preferences.

Recommended security reminders:

- Keep sensitive files out of AI context with .aiignore.

- Prefer Ollama for local/private code review when available.

- Review generated code before inserting or saving it.

- Run npm audit and the project security test before packaging releases.

v0.6.0 - AI Help Cleanup and Output Routing

AI Help was simplified to avoid confusion. AI Help is minimized by default and opens from the upper-right AI Help button. The separate floating minimized AI tab and extra editor AI side panel were removed. The Manual button is separate from AI Help.

Output routing rule:

Questions asked inside AI Help show responses inside AI Help. Coding-focused output from project-aware actions is routed to the AI Coding Assistant workspace page so longer AI content does not crowd normal navigation screens.

v0.6.1 - Cumulative Manual Policy

This release corrects the manual strategy. The app now opens a stable cumulative manual file instead of treating each version as a replacement manual. The original v0.5.5 manual remains at the beginning, and these addendum pages document later changes.

Packaging expectation going forward:

- Do not rewrite the original manual pages unless there is a true correction needed.

- Add a new version addendum section for each release.

- Keep a stable app-facing file named DiligentCodeStudio_UserManual.pdf.

- Keep versioned archive copies for release records.

Version Addendum - v0.6.2 Web Builder + Hosting Tools

This addendum extends the cumulative Diligent Code Studio User Manual. The original manual remains at the beginning of the document; new version-specific material is appended here.

Purpose of this release

v0.6.2 adds a dedicated Web Builder workspace area for users who want to create, preview, host, and deploy websites directly from Diligent Code Studio. The goal is to make web work feel guided instead of forcing the user to remember every command.

New area	Web Builder workspace page added to the main Workspace Menu.
Local hosting	Buttons for local dev server, LAN preview, production build, and production preview.
Global hosting	Buttons to prepare or run Vercel and Netlify preview/production deployment commands.
Installable tools	One-click prepared installs for React Router, Tailwind CSS, Bootstrap, Lucide Icons, Framer Motion, Recharts, ESLint/Prettier, Vercel CLI, and Netlify CLI.
Templates	Static Website template improved and a new React + Vite Website template added.
Setup	Setup & Dependencies now includes Web Builder tools such as Vercel CLI, Netlify CLI, and pnpm.
Tool Registry	Built-in Web Builder and Web Deployment commands added to Tool Registry.

Using the Web Builder page

1. Open or create a web project. Use the Workspace panel to choose a folder, or use Project Templates to create a Static Website or React + Vite Website.

2. Install components. Use the Installable Web Components and Tools cards to prepare or run package installation commands in the active workspace.

3. Host locally. Use Local dev server for normal work on the same computer.

4. Test on the local network. Use LAN preview server when a phone or another PC needs to view the site on the same network.

5. Build production output. Use Production build to create optimized static output in dist/ for Vite projects.

6. Preview production output. Use Production preview before deploying globally.

7. Deploy globally. Use Vercel or Netlify deployment cards after you have logged in to the matching provider CLI.

Command Reference

Local dev server	npm run dev -- --host 127.0.0.1
LAN dev server	npm run dev -- --host 0.0.0.0
Production build	npm run build
Production preview	npm run preview -- --host 127.0.0.1
Vercel preview deploy	npx vercel
Vercel production deploy	npx vercel --prod
Netlify preview deploy	npx netlify-cli deploy
Netlify production deploy	npx netlify-cli deploy --prod

Important notes

- Prepared commands do not modify files until the user runs them. Install and deploy commands may change package.json, package-lock.json, or provider configuration folders.

- Global deployment commands may require account login, project linking, and DNS/custom-domain configuration outside Diligent Code Studio.

- Use AI Help for quick navigation questions. Use AI Coding Assistant for deeper project-specific web build, deployment, and troubleshooting prompts.

- For production hosting, review generated output, secrets, environment variables, contact forms, license notices, analytics scripts, and custom domain settings before publishing.

Suggested Web Builder Workflow

Create a React + Vite Website template, run npm install, start the local dev server, add components such as React Router or Tailwind, run production build, preview the dist output, commit to Git, then deploy with Vercel, Netlify, GitHub Pages, or your own host.

Version Addendum - v0.6.3

Rust Template Build Fix

This addendum documents a targeted fix for a Rust/Tauri build failure in the v0.6.2 Web Builder release. The original manual content remains unchanged at the beginning of this cumulative manual, and this page is appended as the v0.6.3 update.

What was fixed

Area	Update
Web Builder static website template	Corrected the Rust raw string used to generate index.html so embedded anchor links such as **href="#content"** no longer break Rust parsing.
Tauri/Rust build	The compiler error about unknown prefixes such as **grid** and **js** is resolved by using a safer two-hash raw string delimiter.
Version sync	Updated visible app, package, Tauri, and Cargo references to v0.6.3.

Why it happened

The generated HTML contained **href="#content"**. In Rust, a one-hash raw string ends when the compiler sees the sequence quote-hash. The anchor link accidentally created that sequence inside the HTML template. The fix changes the template to a two-hash raw string so the HTML remains valid text inside Rust.

Recommended validation

After extracting the v0.6.3 package, run:

```
npm ci
npm run validate
npm run build
npm run check:rust
npm run tauri:build
```

The sandbox used to prepare this package can validate the frontend build but cannot execute Cargo because Rust/Cargo is not installed there. The Rust fix is targeted directly at the compiler error reported from src-tauri/src/main.rs.